Premium Notes

By @codewithanush

DEPLOYMENT STRATEGIES

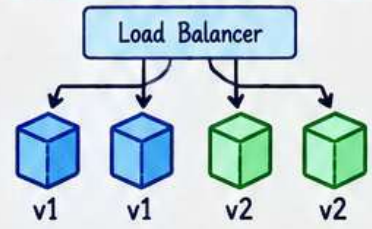
Complete Handwritten Notes

Concepts | Types | Architecture | Real-World Examples
CI/CD Integration | Kubernetes | AWS | Best Practices

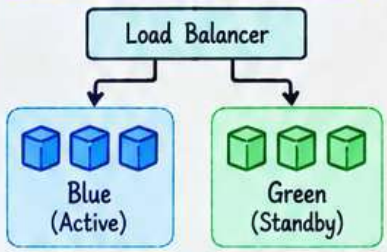
- [x] Introduction to Deployment
- [x] Recreate Deployment
- [x] Rolling Deployment
- [x] Blue-Green Deployment
- [x] Canary Deployment
- [x] A/B & Shadow Deployment
- [x] Kubernetes Deployment
- [x] CI/CD Deployment
- [x] Rollback & Zero-Downtime
- [x] Real-World Scenarios



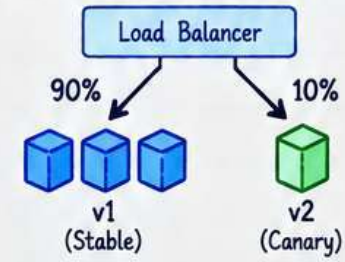
Rolling Deployment



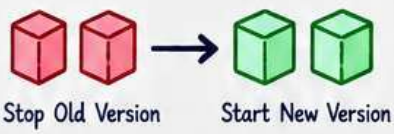
Blue-Green Deployment



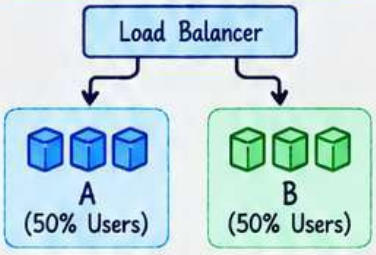
Canary Deployment



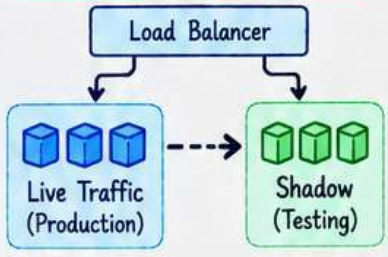
Recreate Deployment



A/B Deployment



Shadow Deployment



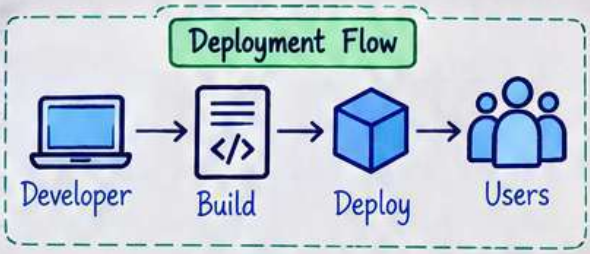
aws AWS | Kubernetes | Docker | Jenkins

10 Pages Premium Notes

1. Introduction to Deployment Strategies

1. What is Deployment ?

Deployment means releasing a new version of an application to a production or target environment so that users can access and use it. It includes copying the application code, configuring required resources, and making it live for users.



2. What are Deployment Strategies ?

Deployment strategies are different methods used to deploy a new version of an application to production with minimal downtime and risk. They help in managing how the new version is released to users, how traffic is handled, and how rollback is performed if any issue occurs.

Key Factors

- ☑ Minimize Downtime
- ☑ Reduce Risk
- ☑ Ensure High Availability
- ☑ Easy Rollback
- ☑ Better User Experience

3. Why are Deployment Strategies Important ?

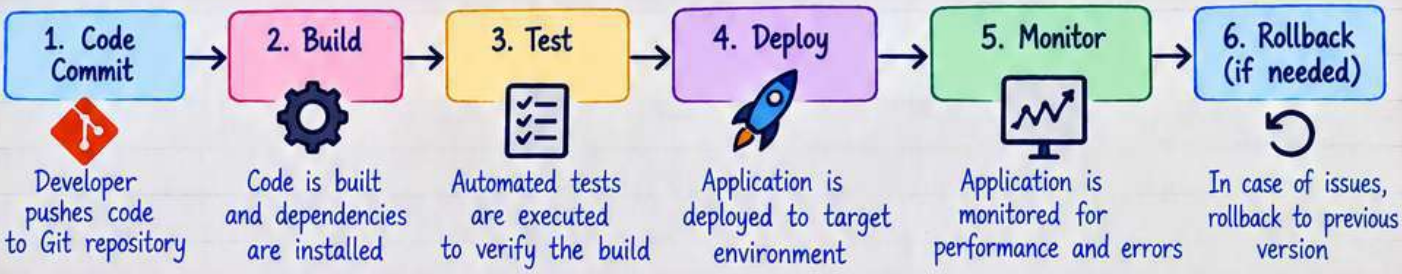
- Helps in releasing new features without service interruption
- Reduces the risk of application failure
- Ensures application availability and stability
- Allows quick rollback in case of issues
- Supports continuous delivery and faster releases
- Improves user experience and business reliability

Without Strategy vs With Strategy

Without Strategy	With Strategy
✗ High Downtime	✓ Minimal Downtime
✗ More Risk	✓ Controlled Release
✗ Difficult Rollback	✓ Easy Rollback
✗ Directly affects all users	✓ Better Stability
	✓ Improved User Experience

4. Deployment Lifecycle

The deployment process follows a series of steps from code commit to production release.



5. Common Deployment Strategies

There are multiple deployment strategies used in real-world based on application requirements.

1. Recreate Deployment
- Stop old version and start new version
- Simple but causes downtime

2. Rolling Deployment
- Gradually replaces old version with new version
- No downtime, widely used

3. Blue-Green Deployment
- Two environments (Blue & Green)
- Switch traffic after testing
- Quick rollback

4. Canary Deployment
- Deploy to small subset of users
- Gradually increase traffic
- Reduces risk

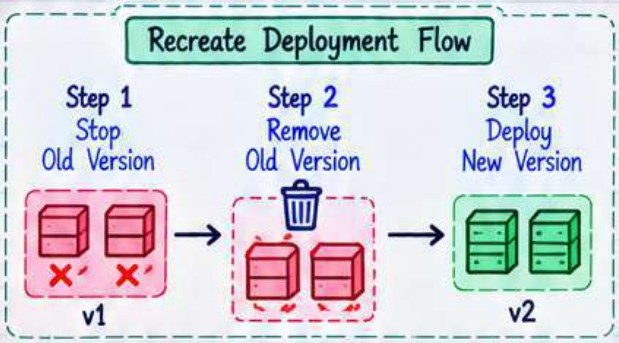
5. A/B Deployment
- Route different users to different versions
- Used for feature testing

6. Shadow Deployment
- New version runs in parallel without impacting users
- Used for testing and validation

2. Recreate Deployment

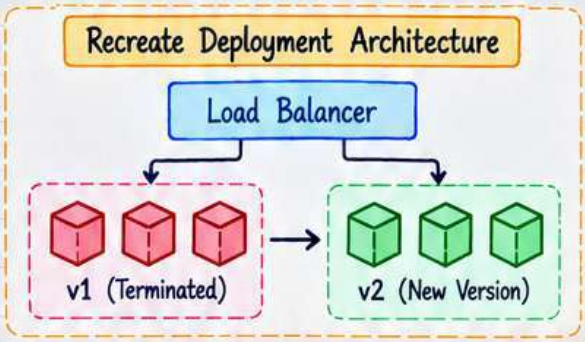
1. What is Recreate Deployment ?

Recreate Deployment is a simple deployment strategy where the existing version of the application is completely stopped and removed, and then the new version is deployed. It does not keep old and new versions running at the same time. This results in a short downtime during deployment.



2. How Does Recreate Deployment Work ?

1. All running instances of the current version are terminated.
2. The existing application is removed completely.
3. New version of the application is deployed.
4. New instances are started and made live.
5. Users can access the application after the new version is fully deployed.



3. Use Cases

- Non-critical applications
- Development and testing environment
- Applications that can tolerate downtime
- Simpler applications with low traffic
- Quick deployments where high availability is not a priority

4. Advantages

- Simple and easy to implement
- No need for complex configuration
- Less resource usage (compared to blue-green/canary)
- Lower cost
- Suitable for small applications

5. Limitations

- Causes downtime
- Users cannot access application during deployment
- Not suitable for high-traffic or critical applications
- Can lead to loss of active sessions

6. Example: Recreate Deployment in Kubernetes

Deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  strategy:
    type: Recreate   # Deployment strategy
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: myapp:v2
        ports:
        - containerPort: 8080
```

Key Configuration

- `strategy.type: Recreate` → Sets the deployment strategy
- `replicas: 3` → Number of instances for new version
- Old pods are terminated before new pods are created
- This results in a temporary downtime during deployment

7. Real-World Example

- A small internal web application used by the development team can use Recreate deployment.
- During deployment, the old version is stopped and removed, and the new version is deployed.
- Since the application is non-critical and has low users, short downtime is acceptable.

8. Important Points

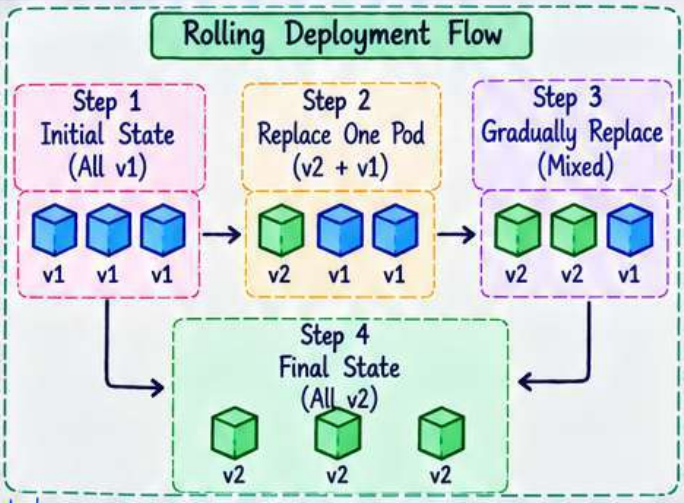
- Recreate deployment should not be used for production applications with high traffic.
- Ensure proper maintenance window if downtime is not acceptable.
- Use health checks after deployment to confirm the new version is running.

3. Rolling Deployment

1. What is Rolling Deployment ?

Rolling Deployment is a deployment strategy where

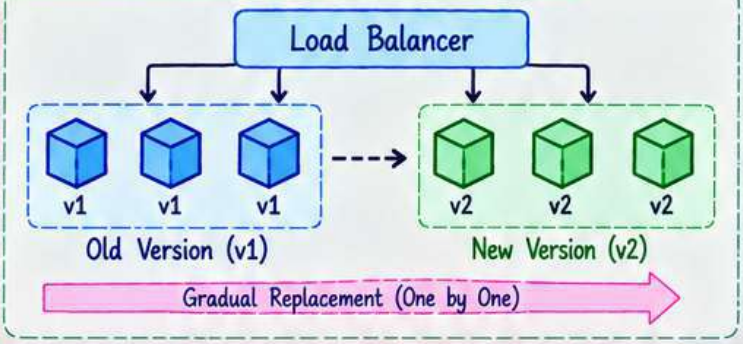
- The new version of the application is gradually rolled out by replacing old instances one by one.
- At no point the entire application is down.
- Both old and new versions run simultaneously during the deployment.
- It ensures zero or minimal downtime.



2. How Does Rolling Deployment Work ?

1. New version is created and deployed.
2. One or a few instances of the old version are terminated.
3. New instances with the new version are started.
4. Traffic continues to flow to the available instances.
5. The process is repeated until all old instances are replaced with the new version.

3. Rolling Deployment Architecture



4. Advantages

- ✅ Zero or minimal downtime
- ✅ Ensures high availability
- ✅ Suitable for most applications
- ✅ Allows monitoring during deployment
- ✅ Risk is reduced as traffic is distributed

5. Limitations

- ❌ Both old and new versions run simultaneously (increases resource usage)
- ❌ Not suitable for database schema changes
- ❌ Rollback can be slower compared to blue-green
- ❌ Requires application to be backward compatible

6. Kubernetes Rolling Deployment Example

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app                    # Deployment strategy
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1        # Number of extra pods to create
      maxUnavailable: 1  # Number of pods to be unavailable
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: myapp:v2
        ports:
        - containerPort: 8080
```

7. Key Configuration Options

Parameter	Description
strategy.type	Defines the strategy (RollingUpdate or Recreate)
rollingUpdate	Configuration for rolling update strategy
maxSurge	Number of extra pods that can be created during the update (default 25%)
maxUnavailable	Number of pods that can be unavailable during the update (default 25%)

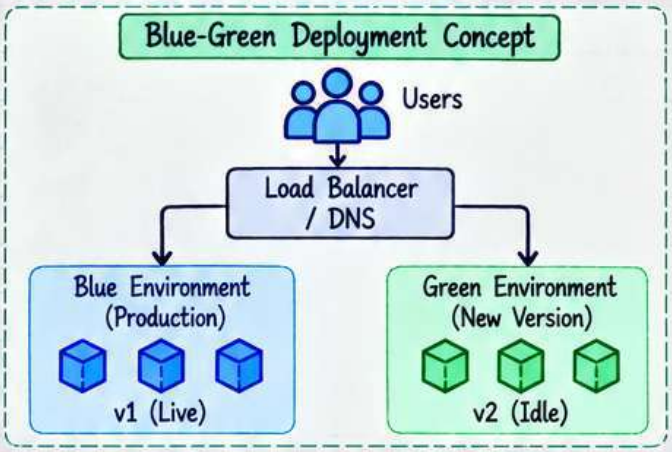
8. Real-World Use Cases

- Web applications with high user traffic
- Microservices deployment
- E-commerce and banking applications
- Applications requiring continuous availability
- When zero-downtime is required

4. Blue-Green Deployment

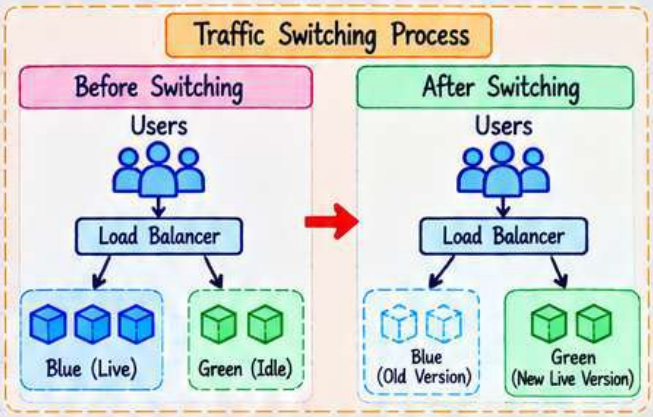
1. What is Blue-Green Deployment ?

Blue-Green Deployment is a deployment strategy where two identical environments are maintained, one for the current live version (Blue) and another for the new version (Green). The new version is deployed and tested in the green environment, and once verified, the traffic is switched from blue to green in a single step.

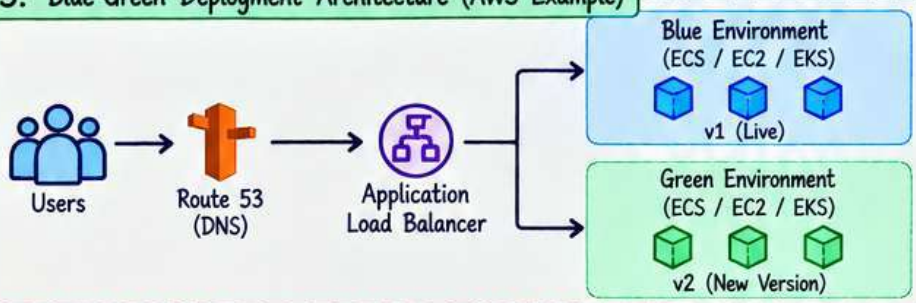


2. How Does Blue-Green Deployment Work ?

1. Create a new environment (Green) identical to the current production environment (Blue).
2. Deploy the new version of the application in the Green environment.
3. Perform testing and validation in the Green environment.
4. Once verified, switch the traffic from Blue to Green using load balancer or DNS.
5. Keep the old environment (Blue) for rollback if needed.



3. Blue-Green Deployment Architecture (AWS Example)



Components:

- Route 53 / Load Balancer
- ECS / EC2 / EKS (Two identical env)
- Auto Scaling (optional)
- Monitoring (CloudWatch)
- CI/CD Pipeline (Jenkins / GitHub Actions)

4. Advantages

- ✅ Zero or minimal downtime
- ✅ Quick and easy rollback
- ✅ Reduced risk
- ✅ Allows full testing in production-like environment
- ✅ Better stability and user experience
- ✅ Suitable for critical applications

5. Limitations

- ❌ Requires double infrastructure (Blue + Green)
- ❌ Higher cost
- ❌ Not suitable for small applications
- ❌ Database schema changes can be complex
- ❌ Resource intensive
- ❌ Requiree proper traffic switching mechanism

6. Real-World Use Cases

- E-commerce applications
- Banking and financial services
- Applications with high availability requirements
- When zero-downtime deployment is required
- Large scale applications
- During major version upgrades

7. Example: Blue-Green Deployment using Kubernetes

Blue Deployment (v1)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-blue
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: my-app
        image: myapp:v1
```

Green Deployment (v2)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-green
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: my-app
        image: myapp:v2
```

8. Important Points

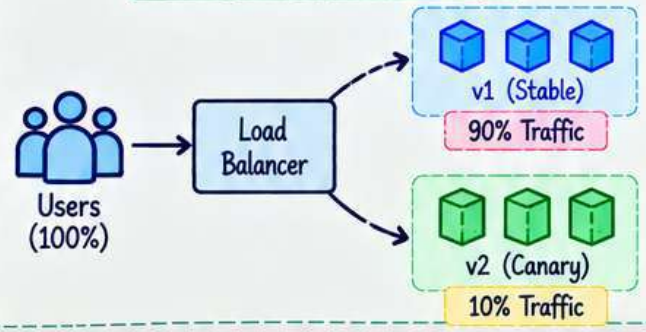
- Both environments should be identical (same infrastructure, configuration, and dependencies).
- Use load balancer or DNS for traffic switching.
- Perform thorough testing in the green environment before switching.
- Keep the blue environment running for rollback.
- Monitor the application after switching.
- This strategy is ideal for critical applications where downtime is not acceptable.

5. Canary Deployment

1. What is Canary Deployment ?

Canary Deployment is a release strategy where the new version of the application is released gradually to a small subset of users instead of all users at once. It helps in testing the new version in a real production environment with minimal risk. If the new version works fine, traffic is gradually increased to all users.

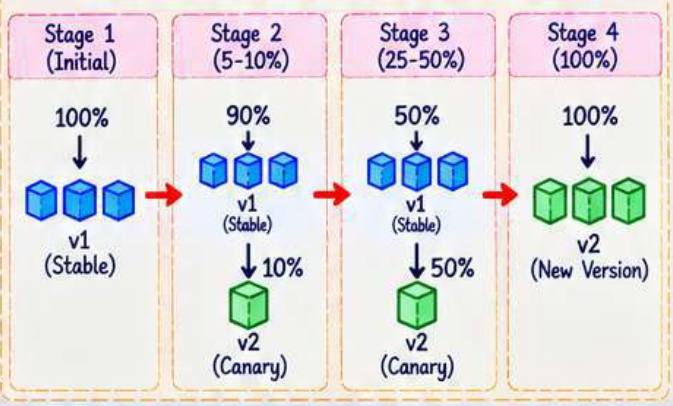
Canary Deployment Concept



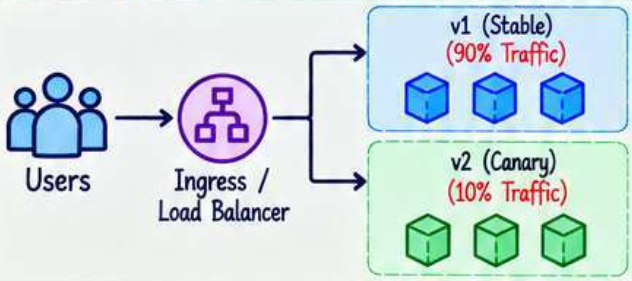
2. How Does Canary Deployment Work ?

1. Deploy the new version to a small percentage of users (e.g. 5% or 10%).
2. Monitor application performance, logs and errors.
3. If the new version works fine, gradually increase traffic (e.g. 25% → 50% → 100%).
4. If any issues are detected, stop the rollout and rollback to the stable version.
5. Once fully tested and verified, the new version replaces the old version.
6. This allows safe and controlled release with minimal risk.

Traffic Flow During Canary Deployment



3. Canary Deployment Architecture (Kubernetes)



4. Advantages

- Reduces risk of application failure
- Real-world testing with live traffic
- Detect issues early
- Gradual rollout and easy rollback
- Minimizes downtime
- Improves application stability
- Better user experience

5. Limitations

- More complex setup and monitoring
- Requires traffic management (e.g. ALB)
- Not suitable for all applications
- Requires proper observability
- Slow rollout process
- Higher operational effort

6. Kubernetes Canary Deployment Example

deployment-v2.yaml (Canary)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-canary
spec:
  replicas: 1  # Small number for canary
  selector:
    matchLabels:
      app: my-app
      version: v2
  template:
    metadata:
      labels:
        app: my-app
        version: v2
    spec:
      containers:
        - name: my-app
          image: myapp:v2
          ports:
          - containerPort: 8080
```

service.yaml (With Traffic Split - Example)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  selector:
    - app: my-app
  ports:
  -- port: 80
   - targetPort: 8080
  type: ClusterIP
```

Traffic Split using Ingress (NGINX Example)

```
nginx.ingress.kubernetes.io/canary: "true"
nginx.ingress.kubernetes.io/canary-weight: "10"
...
  # 10% traffic to canary version
```

7. Real-World Use Cases

- E-commerce applications
- Banking and financial services
- High-traffic web applications
- New feature releases
- Testing performance with real users
- Microservices deployment
- Applications where zero-downtime is required

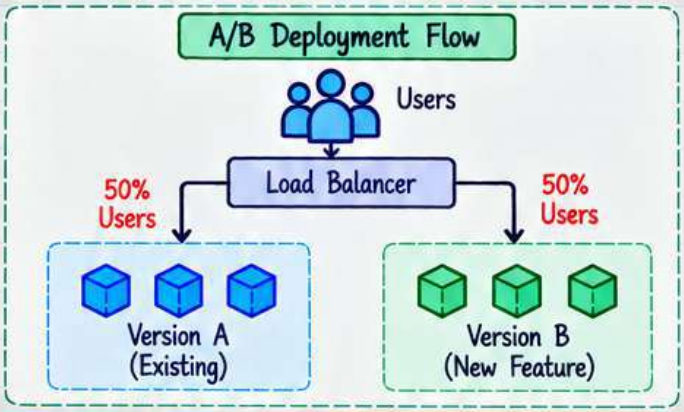
8. Important Points

- Start with a small percentage of traffic (5-10%).
- Use monitoring tools (Prometheus, Grafana, CloudWatch).
- Check application logs, error rates, latency and user behaviour.
- Increase traffic gradually if everything looks good.
- Have a rollback plan in case of issues.
- Use Load Balancer or Ingress for traffic splitting.
- Best for applications with high availability requirements.

6. A/B and Shadow Deployment

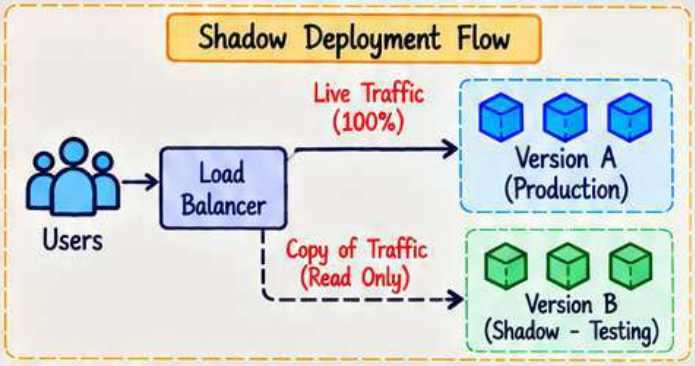
1. What is A/B Deployment ?

A/B Deployment is a release strategy where two versions of an application (A and B) run simultaneously, and different users are routed to different versions. It is mainly used for feature testing, UI changes, or performance comparison to see which version performs better.



2. What is Shadow Deployment ?

Shadow Deployment is a strategy where the new version of the application runs in parallel with the existing version, but real user traffic is not sent to the new version. Instead, a copy of the live traffic is sent to the new version for testing and validation. The responses from the shadow version do not affect the users.



3. A/B Deployment vs Shadow Deployment (Comparison)

Feature	A/B Deployment	Shadow Deployment
Purpose	Compare two versions with real users	Test new version without impacting users
Traffic	Real user traffic is split between A and B	Only a copy of live traffic is sent to new version
User Impact	Users see different versions	Users only see the existing version
Use Case	Feature testing, UI changes, performance comparison	Load testing, validation, monitoring
Risk	Higher (real users use new version)	Lower (no direct impact on users)
Rollback	Switch traffic to stable version	No rollback needed (since users are not using it)
Example	New homepage design, new feature rollout	Testing performance of a new microservice

4. Advantages

A/B Deployment

- ✅ Helps in feature testing
- ✅ Real user feedback
- ✅ Data-driven decision making
- ✅ Improve user experience
- ✅ Can be used for gradual rollout

Shadow Deployment

- ✅ No direct impact on users
- ✅ Safe way to test new version
- ✅ Helps in performance testing
- ✅ Identifies issues before release
- ✅ Useful for large scale applications

5. Limitations

A/B Deployment

- ❌ Requires traffic management
- ❌ Higher risk (real users use new version)
- ❌ Needs monitoring and analysis
- ❌ May require additional infrastructure
- ❌ Complex to implement

Shadow Deployment

- ❌ Requires extra infrastructure
- ❌ Increases resource usage (duplicate traffic)
- ❌ More complex setup
- ❌ Not suitable for all applications
- ❌ Only used for testing (not for feature release)

6. Real-World Use Cases

- **A/B Deployment:**
  - Testing new UI/UX design
  - Comparing different algorithms (e.g. search results)
  - Feature flag testing
- **Shadow Deployment:**
  - Testing a new microservice
  - Performance testing under real traffic
  - Validating database changes
  - Used in large scale systems like e-commerce, banking

7. Important Points

- Use A/B deployment when you want real user feedback.
- Use Shadow deployment when you want to test without impacting users.
- Use monitoring tools (Prometheus, Grafana, CloudWatch) to compare performance and errors.
- Both strategies can be combined with feature flags.
- Ensure proper logging and analytics to analyze results.
- Shadow deployment is commonly used in companies like google, netflix, and amazon for large scale testing.

7. Kubernetes Deployment Strategies

1. Overview

Kubernetes provides different deployment strategies to update applications with minimal downtime. The strategy can be defined in the Deployment resource using the spec.strategy field.

2. Types of Deployment Strategies in Kubernetes

1. RollingUpdate (Default) - gradually replaces old pods
2. Recreate - terminates all old pods and creates new pods
3. Blue-Green - using two separate deployments + service switch
4. Canary - using multiple deployments with traffic split
5. A/B Deployment - using multiple deployments with routing rules
6. Advanced - using service mesh (Istio) for traffic management

3. RollingUpdate Strategy

- Default strategy in Kubernetes.
- Updates pods gradually without downtime.
- Uses maxSurge and maxUnavailable to control rolling update process.

Deployment.yaml (RollingUpdate)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # Extra pod during update
      maxUnavailable: 1 # Pod can be unavailable
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: myapp:v2
```

RollingUpdate Process



4. Recreate Strategy

- Terminates all existing pods first and then creates new pods.
- Simple but causes downtime.
- Used for non-critical applications.

Deployment.yaml (Recreate)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        name: my-app
      - image: myapp:v2
```

5. RollingUpdate Parameters

Parameter	Description
maxSurge	Number of extra pods that can be created during the update. (Default: 25%)
maxUnavailable	Number of pods that can be unavailable during the update. (Default: 25%)
0	Can be set to 0 for more controlled deployment
1	Allows one pod to be added or removed at a time
Percentage	Can be defined as percentage (e.g. 25%) or absolute number (e.g. 2)

6. Effect of maxSurge and maxUnavailable

maxSurge	maxUnavailable	Behaviour
1	1	Default, safe rolling update
0	1	No extra pods, one pod replaced at a time
1	0	Always keep desired replicas running (no downtime)
25%	25%	Kubernetes default values
2	2	Faster update, but higher resource usage

7. Deployment Status and Commands

- Check deployment status:
  `kubectl get deployment`
- Check rollout status:
  `kubectl rollout status deployment/<name>`
- View rollout history:
  `kubectl rollout history deployment/<name>`
- Rollback to previous version:
  `kubectl rollout undo deployment/<name>`
- View deployment details:
  `kubectl describe deployment <name>`

8. Key Points

- RollingUpdate is preferred for most applications.
- Use Recreate only for non-critical applications.
- Use maxSurge and maxUnavailable to control update behaviour.
- Monitor rollout status and application health during deployment.

9. Real-World Use Cases

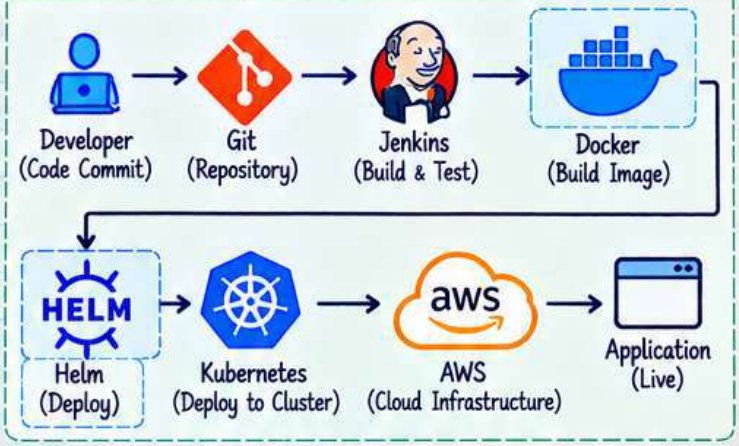
- RollingUpdate - E-commerce, banking, production applications
- Recreate - Internal tools, batch jobs, non-critical apps
- Custom strategies (Blue-Green/Canary) - High availability systems
- Used with Kubernetes service and Ingress for traffic management

8. CI/CD Deployment Strategies

1. What is CI/CD Deployment ?

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is the process of automating the build, test and deployment of applications. It helps in faster and more reliable releases with minimal manual effort using tools like Jenkins, Git, Docker and Kubernetes or cloud providers.

2. CI/CD Deployment Flow (Overview)



3. CI/CD Pipeline Stages

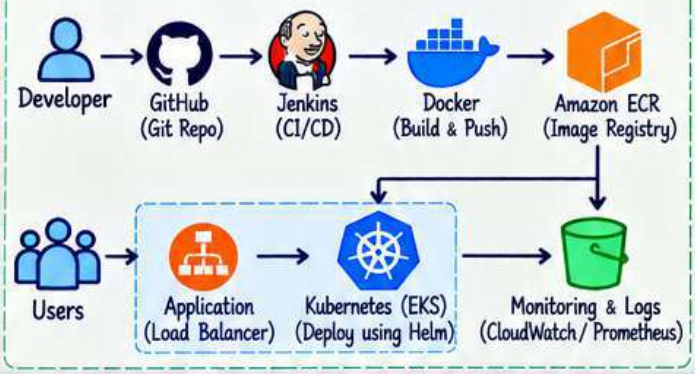
1. **Code Commit** – Developer pushes code to Git.
2. **Build** – Jenkins builds the application.
3. **Test** – Run unit tests, security scans (SonarQube, Trivy).
4. **Package** – Build Docker image and push to registry (e.g. ECR, DockerHub).
5. **Deploy** – Deploy to Kubernetes or AWS using Helm/Terraform/Ansible.
6. **Monitor** – Check application health and logs.

4. Sample Jenkins Pipeline (Deployment)

Jenkinsfile (Declarative Pipeline)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git branch: 'main', url: 'https://github.com/your-repo.git' }
    }
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
    stage('Build Docker Image') {
      steps { sh 'docker build -t my-app:latest .' }
    }
    stage('Push to ECR') {
      steps { sh 'docker push <ecr-repo>/my-app:latest' }
    }
    stage('Deploy to Kubernetes') {
      steps { sh 'kubectl apply -f k8s/deployment.yaml' }
    }
  }
}
```

5. Deployment Architecture (CI/CD with Kubernetes on AWS)



6. Kubernetes Deployment via Helm (Example)

Helm Install Command

```
helm install my-app ./my-app-chart \
  --namespace production \
  --set image.tag=latest \
  --set replicaCount=3
```

Values.yaml (Example)

```
image:
  repository: my-app
  tag:latest
replicaCount: 3
service:
  type: LoadBalancer
  port:80
```

7. AWS Deployment Example (Using EKS + ECR)

1. Build Docker image and push to Amazon ECR.
2. Update Kubernetes manifests with new image tag.
3. Deploy to EKS using kubectl or Helm.
4. Use ALB (Application Load Balancer) for traffic.
5. Monitor using CloudWatch, Prometheus and Grafana.

ECR Login and Push

```
aws ecr get-login-password --region ap-south-1 |
  docker login --username AWS \
    --password-stdin <account-id>.dkr.ecr.ap-south-1.amazonaws.com
docker build -t my-app:latest .
docker tag my-app:latest <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest
docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest
```

8. Advantages

- ✅ Automates entire deployment process
- ✅ Faster and more reliable releases
- ✅ Reduces manual effort and human errors
- ✅ Supports multiple environments (Dev, QA, Prod)
- ✅ Easy rollback to previous version
- ✅ Integrates with testing and security tools

9. Limitations

- ❌ Requires initial setup and maintenance
- ❌ Complex for large applications
- ❌ Need proper security and access control
- ❌ Failures in pipeline can block deployment
- ❌ Requires skilled team and monitoring
- ❌ Cloud costs can increase

10. Real-World Use Cases

- E-commerce and retail applications
- Banking and financial services
- Microservices-based applications
- High-availability production systems
- Automated deployments during sales period
- Multi-environment deployments (Dev → QA → Prod)

9. Rollback & Zero-Downtime Deployment

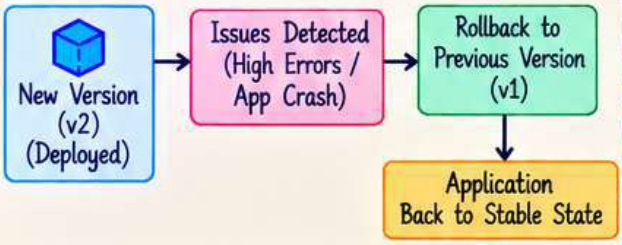
1. What is Rollback ?

Rollback means reverting the application to a previous stable version in case the new deployment has issues. Kubernetes maintains the previous ReplicaSet, which helps to quickly rollback the deployment.

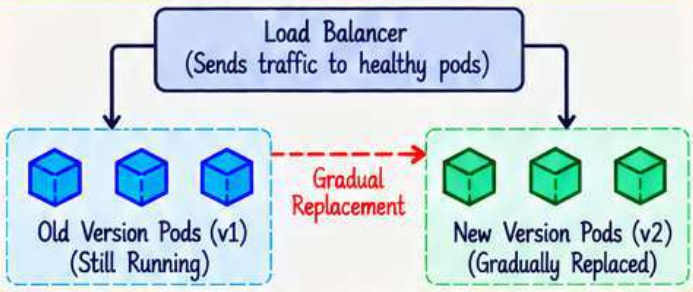
2. What is Zero-Downtime Deployment ?

Zero-downtime deployment ensures that the application remains available to users during the deployment process. It is achieved using strategies like RollingUpdate, Blue-Green or Canary deployment, where the new version is gradually released without stopping the existing version.

3. Rollback Process (Flow)



4. Zero-Downtime Deployment Flow (RollingUpdate)



5. Rollback Commands

Check deployment rollout history

```
kubectl rollout history deployment/my-app
```

See details of a specific revision

```
kubectl rollout history deployment/my-app --revision=2
```

Rollback to previous version

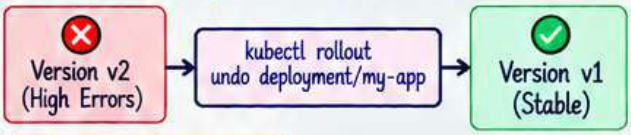
```
kubectl rollout undo deployment/my-app
```

Rollback to a specific revision

```
kubectl rollout undo deployment/my-app --to-revision=2
```

6. Example: Rollback Scenario

Let's say version v2 is deployed and causing high errors. We rollback to the previous stable version (v1).



Rollout History Output

```
$ kubectl rollout history deployment/my-app
 REVISION      CHANGE-CAUSE
  1            Initial deployment
  2            Deployed version v2
  3            Rollback to v1
```

7. Zero-Downtime Deployment Strategies

Strategy	Downtime	How it Works	Use Case
RollingUpdate	No	Gradually replaces old pods	Most common and default
Blue-Green	No	Two separate environments	Production releases
Canary	No	Releases to small % of users	Testing new version with real users

8. Health Checks & Readiness Probes

- Readiness Probe: Ensures pod is ready before receiving traffic.
- Liveness Probe: Restarts pod if application is unhealthy.
- Helps in zero-downtime deployment by sending traffic only to healthy pods.

Example (in deployment.yaml)

```
readinessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 10
  periodSeconds: 5
```

9. Best Practices

- Always maintain multiple replicas (min 2 or 3).
- Use readiness and liveness probes.
- Keep rollback plan ready before deployment.
- Monitor application metrics during rollout.
- Use automated CI/CD pipeline for safe deployment.
- Test new version in staging environment first.

10. Real-World Use Cases

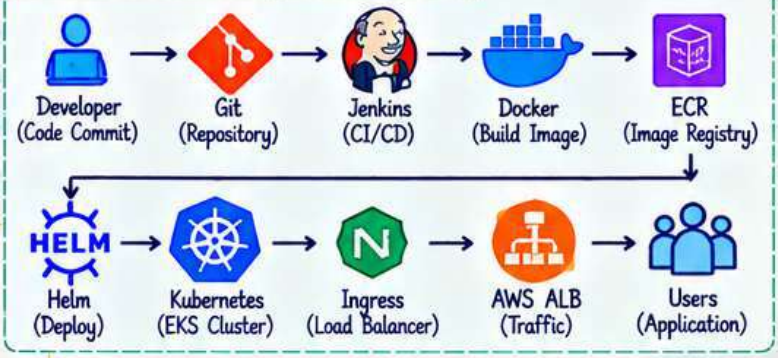
- E-commerce applications (during sales)
- Banking and financial applications
- Microservices-based architecture
- Feature rollout with minimal user impact
- Emergency rollback in case of critical issues
- Used in Dev, QA and Production environments

10. Real-World Deployment Architecture & Scenarios

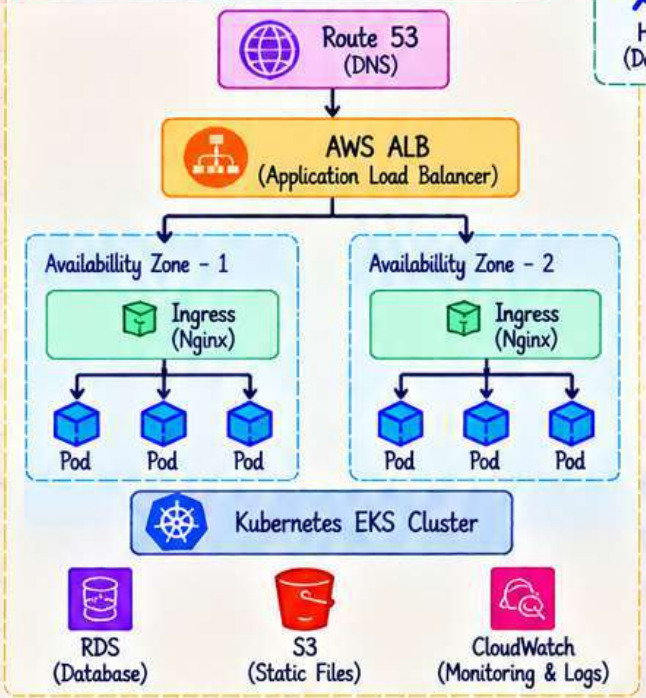
1. Production Deployment Architecture

Below is a real-world production deployment architecture used in most companies (e.g. E-Commerce, Banking, SaaS applications) using CI/CD, Kubernetes and AWS.

2. End-to-End Deployment Flow



3. Kubernetes Production Architecture (on AWS - EKS)



4. Deployment Strategies in Real-World

Strategy	Use Case	Downtime	Description
RollingUpdate	Default, most applications	No	Gradually replaces old pods with new pods
Blue-Green	High-traffic applications	No	Two separate environments (Blue & Green)
Canary	Testing new version	No	Release to small % users and then full rollout
Recreate	Simple applications	Yes	Terminates old pods first and creates new pods

5. E-Commerce High-Traffic Deployment Scenario

- Application: Online shopping platform (handles 50k+ users)
- Strategy Used: Blue-Green or Canary
- Why: To avoid downtime during sales (e.g. Big Billion Days)
- Infra: AWS EKS with Auto Scaling
- Traffic Management: AWS ALB + Ingress (Nginx)
- Monitoring: CloudWatch, Prometheus, Grafana
- Rollback: Automatic if error rate increases

6. Sample Deployment YAML (Production)

deployment.yaml (Production Example)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: 123456789012.dkr.ecr.ap-south-1.amazonaus.com/my-app:v2
        ports:
        - containerPort: 8080
```

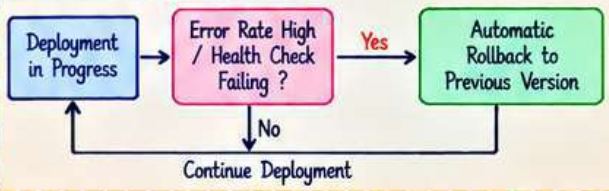
7. Monitoring and Health Checks



Common Health Checks

- Liveness Probe : Restart pod if application is unhealthy
- Readiness Probe : Ensure pod is ready before receiving traffic
- Monitor CPU, Memory, Error rate, Response time
- Set up CloudWatch / Prometheus alerts
- Use auto-scaling based on traffic

8. Rollback Decision Flow



9. Real-World Benefits

- No downtime during releases
- Handles high traffic load
- Automatic rollback on failure
- Better security and stability
- Supports multi-environment setup
- Improved user experience
- Scalable and reliable infrastructure

10. Key Takeaways

- Use RollingUpdate, Blue-Green or Canary for zero-downtime.
- Always maintain multiple replicas.
- Implement proper monitoring and alerts.
- Use auto-scaling for high traffic.
- Keep rollback strategy ready.
- Follow best practices for production deployment.